



指南 > 应用服务 > Location Kit（位置服务） > 获取设备的位置信息开发指导 > 获取设备的位置信息开发指导（ArkTS）

获取设备的位置信息开发指导（ArkTS）

更新时间: 2025-12-26 14:19



场景概述

开发者可以调用HarmonyOS位置相关接口，获取设备实时位置，或者最近的历史位置，以及监听设备的位置变化。

对于位置敏感的应用业务，建议获取设备实时位置信息。如果不需要设备实时位置信息，并且希望尽可能的节省耗电，开发者可以考虑获取最近的历史位置。

接口说明

获取设备的位置信息所使用的接口如下，详细说明参见：[Location Kit API参考](#)。

本模块能力仅支持WGS-84坐标系，如需转换成其他坐标系，请参考[坐标转换工具](#)。

表2 获取设备的位置信息接口介绍

接口名	功能描述
on(type: 'locationChange', request: LocationRequest ContinuousLocationRequest, callback: Callback<Location>): void	开启位置变化订阅，并发起定位请求。

接口名	功能描述
<code>off(type: 'locationChange', callback?: Callback<Location>): void</code>	关闭位置变化订阅，并删除对应的定位请求。
<code>getCurrentLocation(request: CurrentLocationRequest SingleLocationRequest, callback: AsyncCallback<Location>): void</code>	获取当前位置，使用callback回调异步返回结果。
<code>getCurrentLocation(request?: CurrentLocationRequest SingleLocationRequest): Promise<Location></code>	获取当前位置，使用Promise方式异步返回结果。
<code>getLastLocation(): Location</code>	获取最近一次定位结果。
<code>isLocationEnabled(): boolean</code>	判断位置服务是否已经开启。

>

开发步骤

1. 获取设备的位置信息，需要有位置权限，位置权限申请的方法和步骤见[申请位置权限开发指导](#)。
2. 导入geoLocationManager模块，所有与基础定位能力相关的功能API，都是通过该模块提供的。

```
import { geoLocationManager } from '@kit.LocationKit';
```

3. 调用获取位置接口之前需要先判断位置开关是否打开。
查询当前位置开关状态，返回结果为布尔值，true代表位置开关开启，false代表位置开关关闭，示例代码如下：

```
import { geoLocationManager } from '@kit.LocationKit';
try {
  let locationEnabled = geoLocationManager.isLocationEnabled();
} catch (err) {
  console.error("errorCode:" + err.code + ", message:" + err.message);
}
```

}

如果位置开关未开启，可以拉起全局开关设置弹框，引导用户打开位置开关，具体可参考[拉起全局开关设置弹框](#)。

4. 单次获取当前设备位置。多用于查看当前位置、签到打卡、服务推荐等场景。

- 方式一：获取系统缓存的最新位置。

如果系统当前没有缓存位置会返回错误码。

推荐优先使用该接口获取位置，可以减少系统功耗。

如果对位置的新鲜度比较敏感，可以先获取缓存位置，将位置中的时间戳与当前时间对比，若新鲜度不满足预期可以使用方式二获取位置。

```
import { geoLocationManager } from '@kit.LocationKit';
import { BusinessError } from '@kit.BasicServicesKit'
try {
    let location = geoLocationManager.getLastLocation();
} catch (err) {
    console.error("errCode:" + JSON.stringify(err));
}
```

- 方式二：获取当前位置。

首先需要实例化`LocationKit`对象，用于告知系统该应用需要提供何种

筛选目录下的文档标题

- Account Kit（华为账号服务）
- Ads Kit（广告服务）
- AppGallery Kit（应用市场服务）
- App Linking Kit（应用链接服务）
- Call Service Kit（通话服务）
- Cloud Foundation Kit（云开发服务）
- Calendar Kit（日历服务）
- Contacts Kit（联系人服务）
- Enterprise Space Kit（企业数字空间服务）
- File Manager Service Kit（文件管理服务）

■ 设置`LocatingPriority`:

如果对位置的返回精度要求较高，建议`LocatingPriority`参数优先选择`PRIORITY_ACCURACY`，会将一段时间内精度较好的结果返回给应用。

如果对定位速度要求较高，建议`LocatingPriority`参数选择`PRIORITY_LOCATING_SPEED`，会将最先拿到的定位结果返回给应用。

两种定位策略均会同时使用GNSS定位和网络定位技术，以便在室内和户外场景下均可以获取到位置结果，对设备的硬件资源消耗较大，功耗也较大。

■ 设置`locatingTimeoutMs`:

因为设备环境、设备所处状态、系统功耗管控策略等的影响，定位返回的时间会有较大波动，建议把单次定位超时时间设置为10秒。

以快速定位策略(`PRIORITY_LOCATING_SPEED`)为例，调用方式如下:

本文导读

- 场景概述
- 接口说明
- 开发步骤

- ▶ Game Service Kit（游戏服务）
- ▶ Health Service Kit（运动健康服务）
- ▶ IAP Kit（应用内支付服务）
- ▶ Live View Kit（实况窗服务）
- ▼ Location Kit（位置服务）
 - Location Kit简介

附录

▶ Map Kit（地图服务）

```
import { geoLocationManager } from '@kit.LocationKit';
import { BusinessError } from '@kit.BasicServicesKit'
let request: geoLocationManager.SingleLocationRequest = {
  'locatingPriority': geoLocationManager.LocatingPriority.PRIORITY_LOW,
  'locatingTimeoutMs': 10000
}
try {
  geoLocationManager.getCurrentLocation(request).then((result) => { /
    console.log('current location: ' + JSON.stringify(result));
  })
  .catch((error:BusinessError) => { // 接收上报的错误码
    console.error('promise, getCurrentLocation: error=' + JSON.strir
  ));
} catch (err) {
  console.error("errCode:" + JSON.stringify(err));
}
```

通过本模块获取到的坐标均为WGS-84坐标系坐标点，如需使用其它坐标系类型的坐标点， 请进行坐标系转换后再使用。

可使用三方地图提供的SDK能力进行坐标系转换。

5. 持续定位。多用于导航、运动轨迹、出行等场景。

首先要实例化ContinuousLocationRequest对象，用于告知系统该向应用提供何种类型的位置服务，以及位置结果上报的频率。

- 设置locationScenario:
建议locationScenario参数优先根据应用的使用场景进行设置，该参数枚举值定义参见UserActivityScenario，例如地图在导航时使用NAVIGATION参数，可以持续在室内和室外场景获取位置用于导航。
- 设置interval:
表示上报位置信息的时间间隔，单位是秒，默认值为1秒。如果对位置上报时间间隔无特殊要求，可以不填写该字段。

以地图导航场景为例，调用方式如下：

```
import { geoLocationManager } from '@kit.LocationKit';
let request: geoLocationManager.ContinuousLocationRequest= {
  'interval': 1,
  'locationScenario': geoLocationManager.UserActivityScenario.NAVIGATION
}
```

```
let locationCallback = (location:geoLocationManager.Location):void => {
  console.log('locationCallback: data: ' + JSON.stringify(location));
};
try {
  geoLocationManager.on('locationChange', request, locationCallback);
} catch (err) {
  console.error("errCode:" + JSON.stringify(err));
}
```

如果不主动结束定位可能导致设备功耗高，耗电快；建议在不需要获取定位信息时及时结束定位。

```
geoLocationManager.off('locationChange', locationCallback);
```

相关推荐

- | | | | |
|----|------------------|----|------------------|
| 文档 | 位置定位 | 文档 | 持续定位首次定位之后... |
| 文档 | 无法获取设备当前定位信息 | 文档 | 应用后台运行 |
| 文档 | 正地理编码与逆地理编码开发... | 文档 | 获取当前位置信息有一... |
| 文档 | 自动获取定位信息失败 | 文档 | 仓颉-ArkTS 互操作辅... |
| 文档 | 开发指导(ArkTS) | 文档 | 格物开发指导 |

意见反馈

以上内容对您是否有帮助？



意见反馈



如果您有其他疑问，您可以通过开发者社区问答频道来和我们联系探讨。

[社区提问](#) [智能客服提问](#)

简体中文 ▼

